

Digital Design

Chapter 1: Introduction

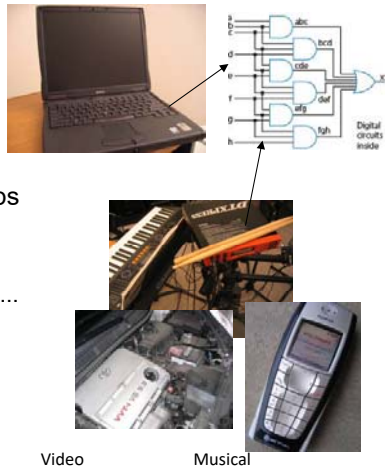
Slides to accompany the textbook Digital Design, with RTL Design, VHDL, and Verilog, 2nd Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2010.
http://www.ddvahid.com

Copyright © 2010 Frank Vahid

*Instructors of courses requiring Vahid's Digital Design textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may **not** be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see http://www.ddvahid.com for information.*

Why Study Digital Design?

- Look "under the hood" of computers
  - Solid understanding --> confidence, insight, even better programmer when aware of hardware resource issues
- Electronic devices becoming digital
  - Enabled by shrinking and more capable chips
  - Enables:
    - Better devices: Better sound recorders, cameras, cars, cell phones, medical devices,...
    - New devices: Video games, PDAs, ...
  - Known as "embedded systems"
    - Thousands of new devices every year
    - Designers needed: Potential career direction



Satellites, DVD players, Video recorders, Musical instruments

Portable music players, Cell phones, Cameras, TVs, ???

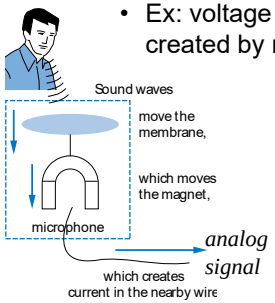
1995 1997 1999 2001 2003 2005 2007

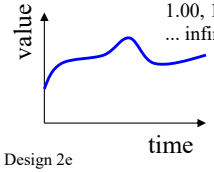
• Years shown above indicate when digital version began to *dominate*
  – (*Not* the first year that a digital version appeared)

Note: Slides with animation are denoted with a small red "a" near the animated items

What Does "Digital" Mean?

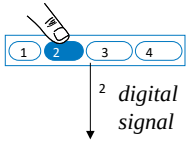
- Analog signal
  - Inifinite possible values
    - Ex: voltage on a wire created by microphone

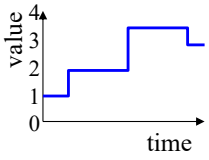




Possible values:
1.00, 1.01, 2.0000009,
... infinite possibilities

- Digital signal
  - Finite possible values
    - Ex: button pressed on a keypad

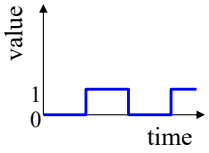




Possible values:
0, 1, 2, 3, or 4.
That's it.

Digital Signals with Only Two Values: Binary

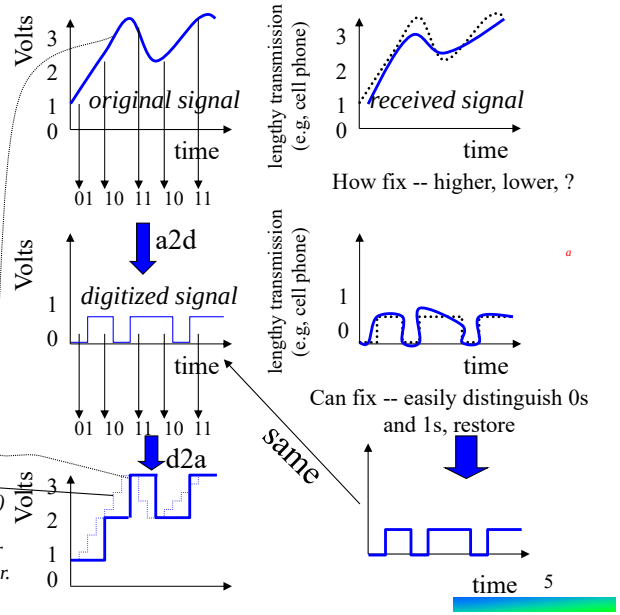
- ***Binary*** digital signal -- only *two* possible values
  - Typically represented as **0** and **1**
  - One *b*inary dig*it* is a ***bit***
  - We'll only consider *binary* digital signals
  - Binary is popular because
    - Transistors, the basic digital electric component, operate using *two* voltages (more in Chpt. 2)
    - Storing/transmitting one of *two* values is easier than three or more (e.g., loud beep or quiet beep, reflection or no reflection)



Example of Digitization Benefit

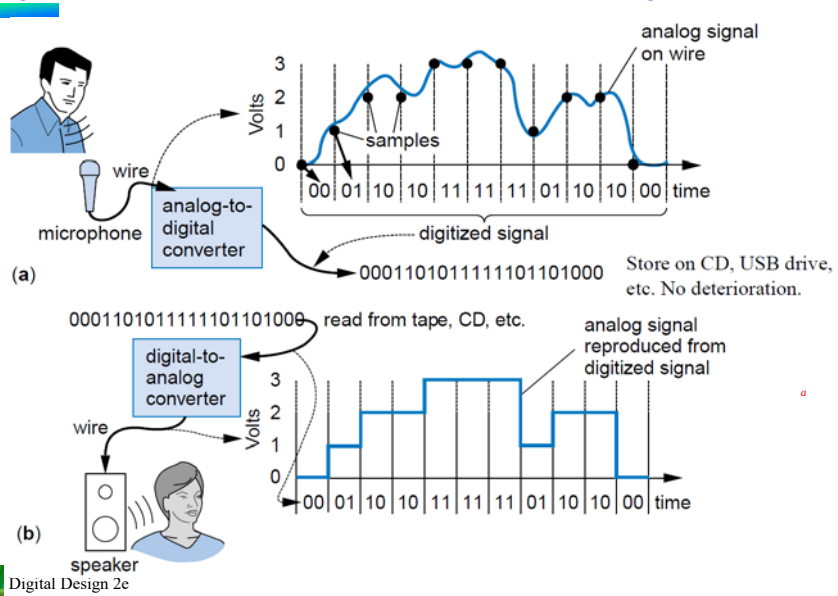
- Analog signal (e.g., audio) may lose quality
  - Voltage levels not saved/copied/transmitted perfectly
- Digitized version enables near-perfect save/cpy/trn.
  - "Sample" voltage at particular rate, save sample using bit encoding
  - Voltage levels still not kept perfectly
  - But we can distinguish 0s from 1s

Let bit encoding be:
1 V: "01"
2 V: "10"
3 V: "11"



Digitized signal not perfect re-creation, (due to A2D and D2A) but higher sampling rate and more bits per encoding brings closer.

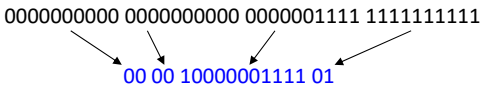
Digitization Benefit: Can Store on Digital Media



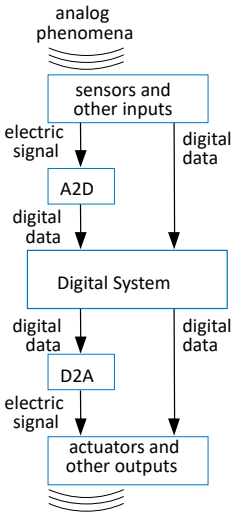
Digitized Audio: Compression Benefit

- Digitized audio can be compressed
  - e.g., MP3s
  - A CD can hold about 20 songs uncompressed, but about 200 compressed
- Compression also done on digitized pictures (jpeg), movies (mpeg), and more
- Digitization has many other benefits too

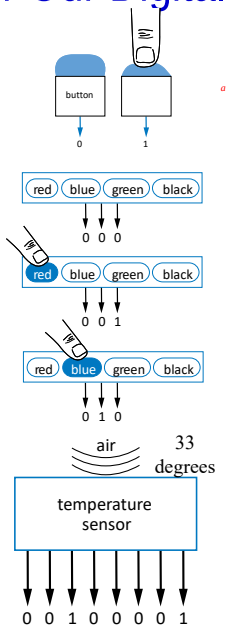
Example compression scheme:
00 --> 0000000000
01 --> 1111111111
1X --> X



How Do We Encode Data as Binary for Our Digital System?



- Some inputs inherently binary
  - Button: not pressed (0), pressed (1)
- Some inputs inherently digital
  - Just need encoding in binary
  - e.g., multi-button input: encode red=001, blue=010, ...
- Some inputs analog
  - Need analog-to-digital conversion
  - As done in earlier slide -- sample and encode with bits



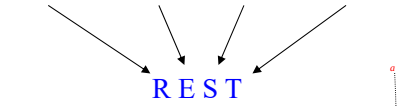
How to Encode Text: ASCII, Unicode

- ASCII: 7- (or 8-) bit encoding of each letter, number, or symbol
- Unicode: Increasingly popular 16-bit bit encoding
  - Encodes characters from various world languages

Symbol	Encoding
R	1010010
S	1010011
T	1010100
L	1001100
N	1001110
E	1000101
0	0110000
.	0101110
<tab>	0001001

Symbol	Encoding
r	1110010
s	1110011
t	1110100
l	1101100
n	1101110
e	1100101
9	0111001
!	0100001
<space>	0100000

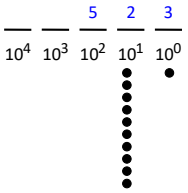
Question:
What does this ASCII bit sequence represent?
1010010 1000101 1010011 1010100

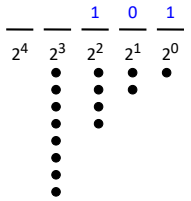


Note: small red "a" (a) in a slide indicates animation

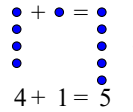
How to Encode Numbers: Binary Numbers

- Each position represents a quantity; symbol in position means how many of that quantity
  - Base ten (***decimal***)
    - Ten symbols: 0, 1, 2, ..., 8, and 9
    - More than 9 -- next position
      - So each position power of 10
    - Nothing special about base 10 -- used because we have 10 fingers
  - Base two (***binary***)
    - Two symbols: 0 and 1
    - More than 1 -- next position
      - So each position power of 2



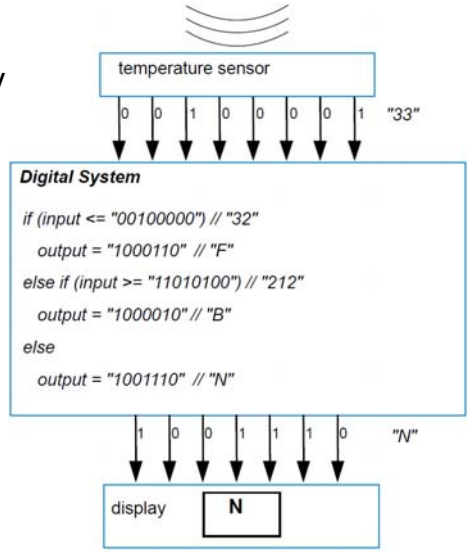


Q: How much?



Using Digital Data in a Digital System

- A temperature sensor outputs temperature in binary
- The system reads the Fahrenheit temperature, outputs ASCII code:
  - "F" for freezing (0-32)
  - "B" for boiling (212 or more)
  - "N" for normal
- A display converts its ASCII input to the corresponding letter



We'll later see how to build circuits to implement desired digital system behavior.

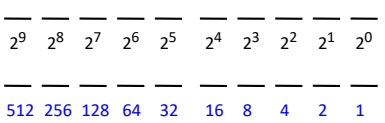
How to Encode Numbers: Binary Numbers

- Working with binary numbers
  - In base ten, helps to know powers of 10
    - one, ten, hundred, thousand, ten thousand, ...
  - In base two, helps to know powers of 2
    - one, two, four, eight, sixteen, thirty two, sixty four, one hundred twenty eight
      - (Note: unlike base ten, we don't have common names, like "thousand," for each position in base ten -- so we use the base ten name)

Practice counting up by powers of 2:

2048 1024 512 256 128 64 32 16 8 4 2 1

Useful to know powers of 2:



- Just add weights
  - 1_2 is just $1*2^0$, or 1_{10}.
  - 110_2 is $1*2^2+ 1*2^1+ 0*2^0$, or 6_{10}. We might think of this using base ten weights: 1*4 + 1*2 + 0*1, or 6.
  - 10000_2 is 1*16 + 0*8 + 0*4 + 0*2 + 0*1, or 16_{10}.
  - 10000111_2 is 1*128 + 1*4 + 1*2 + 1*1 = 135_{10}. Notice this time that we didn't bother to write the weights having a 0 bit.
  - 00110_2 is the same as 110_2 above —the leading 0's don't change the value

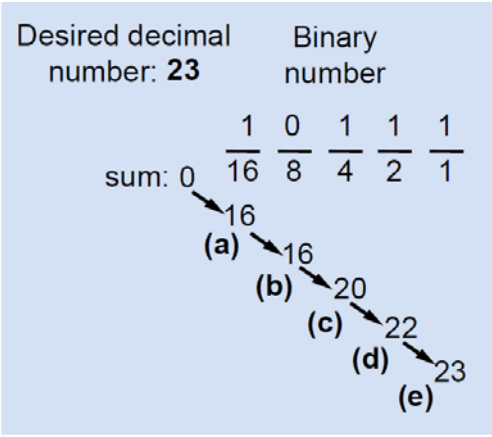
Converting from Decimal to Binary

- Put 1 in leftmost place without sum exceeding number
- Track sum

	Desired decimal number: **12**	Current sum	Binary number
(a)	16 > 12, too big; Put 0 in 16's place	0	**0** / ***16*** 8 4 2 1
(b)	8 <= 12, so put 1 in 8's place, current sum is 8	8	0 / 16 **1** / ***8*** 4 2 1
(c)	8+4=12 <= 12, so put 1 in 4's place, current sum is 12	12	0 / 16 1 / 8 **1** / ***4*** 2 1
(d)	Reached desired 12, so put 0s in remaining places	*done*	0 / 16 1 / 8 1 / 4 0 / 2 0 / 1

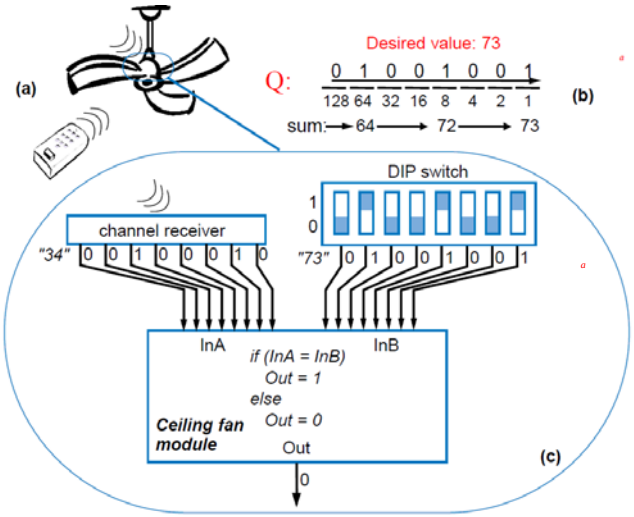
Converting from Decimal to Binary

- Example using a more compact notation

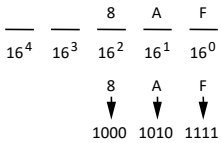


Example: DIP-Switch Controlled Channel

- Ceiling fan receiver should be set in factory to respond to channel "73"
- Convert 73 to binary, set DIP switch accordingly



Base Sixteen: Another Base Used by Designers



hex	binary	hex	binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

- Nice because each position represents four base two positions
  - Compact way to write binary numbers
- Known as ***hexadecimal***, or just ***hex***

Q: Write 11110000 in hex

F 0

Q: Convert hex A01 to binary

1010 0000 0001

Decimal to Hex

- Easy method: convert to binary first, then binary to hex

Convert 99 base 10 to hex

First convert to binary:

0	1	1	0	0	0	1	1
128	64	32	16	8	4	2	1

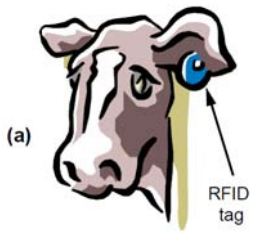
Then binary to hex:

6 3

(Quick check: 6*16 + 3*1 = 96+3 = 99)

Hex Example: RFID Tag

- Batteryless tag powered by radio field
  - Transmits unique identification number
  - Example: 32 bit id
    - 8-bit province number, 8-bit country number, 16-bit animal number
    - Tag contents are in binary
    - But programmers use hex when writing/reading





(c)	Province: 7	City: 160	Animal: 513
(d)	00000111	10100000	00000010 00000001
(e)	07	A0	02 01
(f)	Tag ID in hex: 07A00201		

Converting from Decimal to Binary Numbers: Addition Method (Easy for Humans)

- Goal
  - Get the binary weights to add up to the decimal quantity
    - Work from left to right
    - (Right to left – may fill in 1s that shouldn't have been there – try it).

Desired decimal number: **12**

32	16	8	4	2	1	
1						=32 too much
0	1					=16 too much
0	0	1				=8 ok, keep going
0	0	1	1			=8+4=12 DONE
0	**0**	**1**	**1**	**0**	**0**	answer

Converting from Decimal to Binary Numbers: Subtraction Method (Easy for Humans)

- Subtraction method
  - To make the job easier (especially for big numbers), we can just subtract a selected binary weight from the (remaining) quantity
    - Then, we have a new remaining quantity, and we start again (from the present binary position)
    - Stop when remaining quantity is 0

Remaining quantity: **12**

32	16	8	4	2	1	
1						32 is too much
0	1					16 is too much
0	0	1				**12** – 8 = **4**
0	0	1	1			**4**-4=**0** DONE
0	**0**	**1**	**1**	**0**	**0**	answer

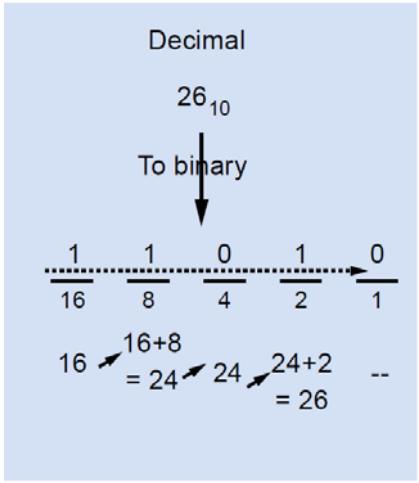
Converting from Decimal to Binary Numbers: Subtraction Method Example

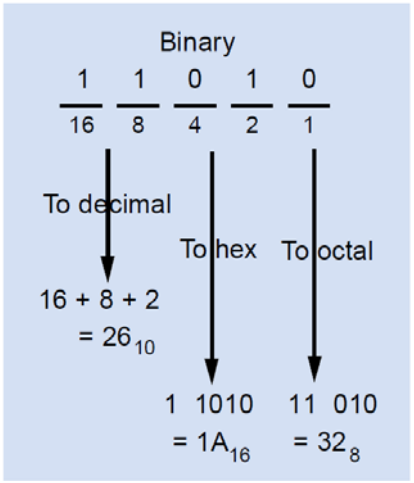
- Q: Convert the number "23" from decimal to binary

A: Remaining quantity	Binary Number (32 16 8 4 2 1)
23	0 0 0 0 0 0
23 − 16 = 7	0 1 0 0 0 0
7 − 4 = 3	0 1 0 1 0 0
	8 is more than 7, can't use
4 − 2 = 1	0 1 0 1 1 0
1 − 1 = 0	0 1 0 1 1 1

→ Done! 23 in decimal is 10111 in binary.

Converting To/From Binary by Hand: Summary





Divide-By-2 Method Common in Automatic Conversion

- Repeatedly divide decimal number by 2, place remainder in current binary digit (starting from 1s column)

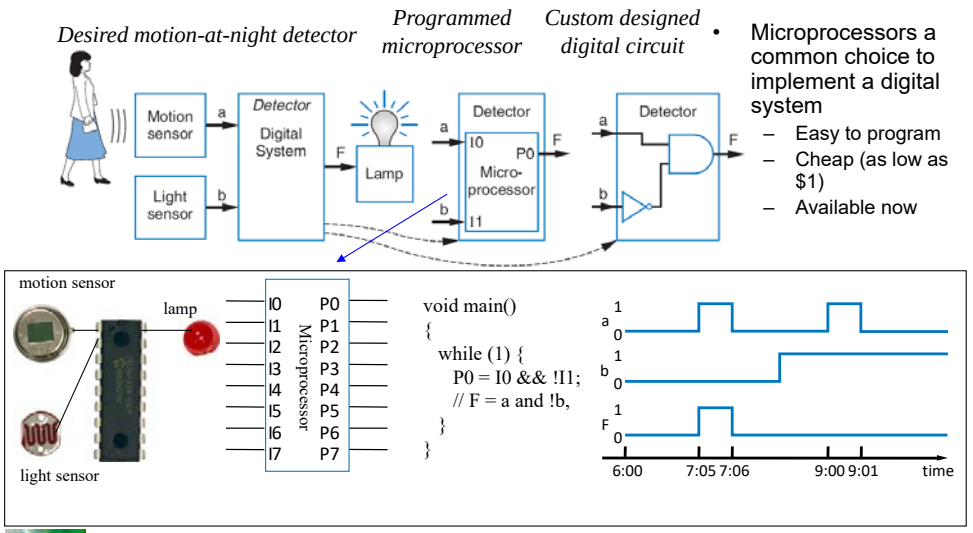
	Decimal	Binary
1. Divide decimal number by 2 Insert remainder into the binary number Continue since quotient (6) is greater than 0	$2\overline{)12}$ (6), −12, (0)	0 1 *(current value: 0)*
2. Divide quotient by 2 Insert remainder into the binary number Continue since quotient (3) is greater than 0	$2\overline{)6}$ (3), −6, 0	0 0 1 *(current value: 0)*
3. Divide quotient by 2 Insert remainder into the binary number Continue since quotient (1) is greater than 0	$2\overline{)3}$ (1), −2, 1	1 0 0 1 *(current value: 4)*
4. Divide quotient by 2 Insert remainder into the binary number Quotient is 0, done	$2\overline{)1}$ (0), −0, 1	1 1 0 0 1 *(current value: 12)*

Note: Works for any base N—just divide by N instead

Bytes, Kilobytes, Megabytes, and More

- Byte: 8 bits 
- Common metric prefixes:
  - kilo (thousand, or 10^3), mega (million, or 10^6), giga (billion, or 10^9), and tera (trillion, or 10^{12}), e.g., kilobyte, or KByte
- BUT, metric prefixes also commonly used inaccurately
  - 2^{16} = 65536 commonly written as "64 Kbyte"
  - Typical when describing memory sizes
- Also watch out for "KB" for kilobyte vs. "Kb" for kilobit

Implementing Digital Systems: Programming Microprocessors Vs. Designing Digital Circuits



- Microprocessors a common choice to implement a digital system
  - Easy to program
  - Cheap (as low as $1)
  - Available now

Digital Design: When Microprocessors Aren't Good Enough

- With microprocessors so easy, cheap, and available, why design a digital circuit?
  - Microprocessor may be too slow
  - Or too big, power hungry, or costly



Wing controller computation task:
- 50 ms on microprocessor
- 5 ms as custom digital circuit

If must execute 100 times per second:
- 100 * 50 ms = 5000 ms = 5 seconds
- 100 * 5 ms = 500 ms = 0.5 seconds

Microprocessor too slow, circuit OK.

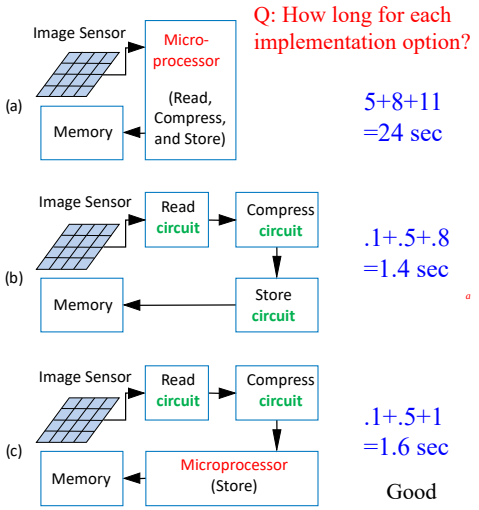
Digital Design: When Microprocessors Aren't Good Enough

- With microprocessors so easy, cheap, and available, why design a digital circuit?
  - Microprocessor may be too slow
  - Or too big, power hungry, or costly



Sample digital camera task execution times (in seconds) on a microprocessor versus a digital circuit:

Task	Microprocessor	Custom Digital Circuit
Read	5	0.1
Compress	8	0.5
Store	1	0.8



Chapter Summary

- Digital systems surround us
  - Inside computers
  - Inside huge variety of other electronic devices (embedded systems)
- Digital systems use 0s and 1s
  - Encoding analog signals to digital can provide many benefits
    - e.g., audio -- higher-quality storage/transmission, compression, etc.
  - Encoding integers as 0s and 1s: Binary numbers
- Microprocessors (themselves digital) can implement many digital systems easily and inexpensively
  - But often not good enough -- need custom digital circuits